

Android Memory Management - Detailed Solutions

Exercise 1:

1. The app will continuously allocate memory until it crashes.
2. Each Android app has a fixed heap limit enforced by ART, independent of total device RAM.
3. ART enforces heap limits by assigning each app a fixed maximum heap size. During allocation, it checks whether the request fits within this limit. If not, it triggers garbage collection, and if memory is still insufficient, it throws an `OutOfMemoryError`, preventing the app from exceeding its allocated heap.
4. Exception: `OutOfMemoryError`

Exercise 2:

1. It creates many short-lived objects → fills memory quickly → triggers GC frequently.
2. Young Generation, where most objects are short-lived.
3. Most objects die in the young generation because they are short-lived and become unreachable shortly after creation. The generational garbage collector is designed based on the observation that most objects are temporary, so they are collected quickly in the young generation before being promoted to the old generation.
4. Causes UI jank (freezing) due to GC pauses exceeding ~16ms.
5. Better alternative, Background thread reduces UI impact.

Exercise 3:

1. Reuses memory instead of creating new objects.

This solution reduces garbage collection frequency because it reuses a single `StringBuilder` object instead of creating new `String` objects in each iteration. This significantly reduces the number of temporary objects allocated in memory, lowering pressure on the young generation and decreasing how often garbage collection is triggered.

In fact, in the unoptimized code (Ex 2), we create a new `String` object in every iteration.

What Happens Internally? Each loop iteration:

- Creates multiple temporary objects:
 - o `StringBuilder` (implicit)
 - o Intermediate `String`
 - o Final `String`
- These objects become unreachable immediately

The Result:

- Heap fills quickly
- Frequent young generation GC cycles

In the optimized code,

- Reduces object creation
- Minimizes GC frequency
- Avoids memory pollution

2. Why This Reduces GC Frequency : Fewer allocations → fewer GC runs.

1. Object Reuse Instead of Creation

- Only one StringBuilder object is created
- Reused across all iterations
- Fewer objects → less garbage

2. Fewer Temporary Allocations

- Avoids repeated creation of intermediate String objects
- Reduces pressure on the young generation

3. Slower Heap Growth

- Memory usage grows much more slowly
- GC is triggered less often

4. Less “Memory Pollution”

- Original code (Ex 2) floods memory with short-lived objects
- Optimized code keeps memory stable

3. Memory pollution = unnecessary temporary objects.

Exercise 4:

Ranking: Foreground > Visible > Service > Background > Cached

App A

We will suppose that the current is the Visible activity onResume() with user interacting UI

So it is Foreground process

App B

Visible activity onStart() + started service

It is a Visible process (could be Foreground if onResume()), Activity dominates everything.

If service started in foreground with startForeground() → it will be considered as Foreground process.

App C (TRICKY ONE)

No visible activity, Only bound service connection

This is NOT service process, NOT foreground process

It is Background process

Important: Bound service alone does NOT elevate process priority

App D

No activity visible, Started service running

It is Service process

This is the ONLY real service process here.

App E

No activity, No service → Cached process

Final Ranking (Highest → Lowest)

1. App A → Foreground process (FG) OR App B (if startForeground())
2. App B → Visible process OR App A if B could be FG with calling startForeground()
3. App D → Service process
4. App C → Background process
5. App E → Cached process

KEY TRICK EXPLAINED

Attention: “It has a service → so it must be service process” is Wrong

Correct rule:

Service type Process impact

startService() YES → Service process possible

bindService() NO → no ranking upgrade

Only started services can elevate an app to service process priority, while bound services only enable interaction and do not affect Android process ranking, which is primarily determined by activity visibility and service state.

Exercise 5:

Process State Table

T1 (Initial launch)

App	State	Process Type
A	Foreground Activity	Foreground
B	Foreground Activity	Visible
C	Foreground Activity	Visible

T2 (Services start / binding begins)

App	State	Process Type
A	Visible Activity + started service	Foreground
B	Foreground Activity + bound service	Foreground
C	Foreground Activity	Foreground

T3 (UI removed)

App	State	Process Type
A	Service only running	Service process
B	Bound service only	Background process
C	No UI, no service	Background process

T4 (System cleanup)

App	State	Process Type
A	Service stopped	Cached
B	Service stopped	Cached
C	Cached only	Cached

FINAL TRACE DIAGRAM (Order)

T1: A(FG) B(Visible) C(Visible)

T2: A(FG+S) B(FG+Bound) C(FG)

T3: A(Service) B(BG) C(BG)

T4: A(Cached) B(Cached) C(Cached)

Attention

1. Started Service keeps process alive → App A stays Service process at T3
2. Bound Service does NOT elevate rank → App B drops to Background once Activity is gone
3. No service So immediate degradation → App C quickly becomes Background → Cached
4. Everything eventually becomes Cached, When no active components remain

Android process priority is dynamically adjusted over time based on active components, where foreground activities dominate initially, started services maintain service-level priority after UI removal, bound services do not affect process priority, and inactive processes eventually degrade into cached state.

Exercise 6:

We will track each app across:

T1 → T2 → T3 → T4

T1: App Launched, At this moment, All apps just started

App	State
A	Foreground (Activity visible with onResume)
B	Foreground (Activity visible with onResume)
C	Foreground (Activity visible with onResume)
D	Service (no UI, started service)
E	Foreground (onStart → onResume)

Ranking (High → Low) is A = B = C = E > D

T2: User Interaction Changes, Now users start leaving apps or interacting.

App	What happened	State
A	User leaves (onStop)	Background
B	Music Started (startForeground)	Foreground
C	Bound to service + onPause	Visible (with Bound Service)
D	Download running	Service
E	User exits (onPause)	Visible

Ranking is B > C = E > D > A

T3: UI Removed / Services Running, Now all Activities are gone.

App	What happened	State
A	nothing running	Cached
B	onStop + music continues	Foreground Service (music continues)
C	onStop + Bound Service continues	Background
D	Still downloading	Service
E	User exits (onStop)	Cached

Important concept: Bound service alone is WEAK, It does NOT keep high priority

Ranking: $B > D > C > A = E$

T4: System Cleanup, Now everything stops.

App	State
A	Cached
B	Cached (music stopped)
C	Cached (service stopped)
D	Cached (service ended)
E	Cached

Ranking $A = B = C = D = E$

All equal $\rightarrow$ all can be killed

FINAL SUMMARY

T1: $A \ B \ C \ E$ (Foreground) $> D$ (Service)

T2: B (Foreground) $> C \ E$ (Visible) $> D$ (Service) $> A$ (Background)

T3: B (FG) $> D$ (Service) $> C$ (BG) $> A = E$ (Cached)

T4: ALL Cached (equal)

Exercise 7:

Kill order:

$E \rightarrow D \rightarrow C \rightarrow B \rightarrow A$

Step 1: Assign Android process priority (OOM importance)

Android LMK kills in this order (lowest first):

1. Cached process (lowest priority)
2. Background process
3. Service process
4. Visible process
5. Foreground process (highest priority)

Step 2: Classify each app

App	Type	Priority Level
A	Foreground	Very High
B	Foreground	Very High
C	Foreground	Very High
D	Background	Low
E	Cached	Lowest

Step 3: LMK decision logic

LMK always chooses: Lowest priority process FIRST

If tie → smallest memory impact OR least important process

Step 4: Who gets killed?

Candidates:

App E (Cached)

- 80 MB
- Lowest priority

FIRST CHOICE for killing

App D (Background)

- 120 MB
- Slightly higher priority than cached

Apps A, B, C (Foreground)

- Protected unless extreme memory pressure

FINAL LMK ACTION

KILL → App E (Cached process, 80 MB)

Step 5: After killing App E

Now free memory = 80 MB

Still not enough for App F (needs 200 MB)

LMK continues to select Next candidate which is App D (Background, 120 MB)

Now killing App D gives Total freed = 200 MB

FINAL SYSTEM RESULT

Step 1: Kill App E (Cached)

Step 2: Kill App D (Background)

→ App F can now launch

Very important:

1. Priority decides WHO dies first, Not memory size alone.
2. Cached processes are always first victims, Even if they use little memory.
3. Foreground apps are protected, Even if they use more memory.
4. LMK is a priority-first, memory-second system, Order matters more than RAM usage.

The Android Low Memory Killer selects processes for termination based primarily on process priority (cached → background → service → Visible → foreground), and only then considers memory usage, ensuring lower-priority processes are killed first to free memory for new high-priority app launches.

Exercise 8:

1.

App	Component	OOM adj score	Dirty cost
A	Foreground Activity	-17	Low
B	Foreground Activity + Started Service	-17	Very high
C	Background Service only (not bound, music playback, no UI)	6	Very Low
D	Background Activity (not visible)	12	Very high
E	Cached process (no components active)	15	Very Low

2. LMK selection?

Step 1: Sort by OOM priority

Lowest importance first:

App E → 15 (cached)

App D → 12 (background)

App C → 6 (service)

App A → -17 (foreground)

App B → -17 (foreground)

Step 2: Add Dirty Page Cost Factor

We define a simple rule:

Higher dirty pages = more expensive to kill → LMK avoids unless necessary

App	OOM	Kill Cost
-----	-----	-----------

E	15	VL
---	----	----

D	12	VH
---	----	----

C	6	VL
---	---	----

A	-17	L
---	-----	---

B	-17	VH
---	-----	----

Step 3: LMK Decision Logic

LMK prefers:

LOW OOM score FIRST

LOW dirty pages FIRST

MINIMAL write-back cost FIRST

Step 4: Kill Order Simulation

Step 1: Kill App E: Cached, Very Low dirty pages cost → Cheapest kill

Freed: 90 MB but Still not enough → App F needs 250 MB

Step 2: Kill App C OR D?

Compare:

App C: Service process (OOM 6), Very Low dirty pages cost

App D: Background process (OOM 12), Very HIGH dirty pages cost

Even though D is lower priority than C, but D is more expensive to kill

So, LMK prefers to Kill App C next (lower OOM first) → Freed: +180 MB

Total freed: $90 + 180 = 270$ MB

App F can now launch

FINAL KILL ORDER

1. App E (Cached, cheapest kill)
2. App C (Service, low dirty cost)

Remarks:

1. LMK is NOT just priority-based, It is: Priority + memory reclaim cost (dirty pages)
2. Cached processes die first, Even if clean or dirty
3. Dirty pages increase “kill cost”, LMK avoids high dirty-memory apps unless necessary
4. Foreground apps are protected, Even with low dirty pages.

Android Low Memory Killer selects processes based on OOM priority first and memory reclaim cost (including dirty pages), preferring to terminate cached and low-priority processes with minimal write-back overhead before considering service or foreground processes.

3. Score and LMK decision?

Step 1: We compute Final score per app

Final Score = OOM_ADJ + Memory_Weight + Dirty_Page_Penalty

App	Memory (MB)	Dirty pages (MB)	OOM adj score	Memory weight	Dirty page penalty	Score
A	200	20	-17	4	2	-11
B	240	80	-17	4.8	8	-4.2
C	180	10	6	3.6	1	10.6
D	150	90	12	3	9	24
E	90	5	15	1.8	0.5	17.3

Step 2: Final Kill Ranking (Highest SCORE FIRST)

1. App D → 24 (easiest to kill)
2. App E → 17.3
3. App C → 10.6
4. App B → -4.2
5. App A → -11 (hardest to kill, safest foreground-heavy workload BUT B has highest dirty cost!!!!)

Step 3: LMK Decision (App F needs memory)

Assume App F needs 250 MB RAM → system must free memory.

First kill target = highest score

App D (Score 24): Background process, High dirty pages (expensive but acceptable), Best candidate

After killing App D → Freed: 150 MB (minus dirty write-back cost delay) but Still not enough → App F needs more.

Next target = App E (17.3), Cached process, Very cheap to kill

After killing App E → Freed: +90 MB but Still not enough → App F needs more.

Next target = App C (10.6), Service process, Very cheap to kill

After killing App C → Freed: +180 MB.

App F can now launch

Remarks:

1. Android does NOT use a simple list. It uses a score-based heuristic system
2. Dirty pages matter economically. High dirty = high “write-back cost”
3. Cached + Background are first victims. Even if memory difference is small
4. Foreground apps survive even if large. Because OOM score is very negative

Android memory management uses an OOM score system combining process priority, memory footprint, and dirty page cost to compute a final kill score, where processes with higher scores (typically cached and background apps with high reclaim cost) are selected first by Low Memory Killer under memory pressure.

Exercise 9:

1. Callback to notify app of low memory conditions and is called before system kills processes.
2. Gradual cache cleaning. It adds cache clearing at different trim levels (light → aggressive)
3. LMK (Low Memory Killer / Android LMKD on modern systems) kills processes based on memory pressure and process priority.

Calling `onTrimMemory()` and releasing memory:

- reduces your RSS (resident memory usage)
- lowers your process's pressure contribution
- makes your app appear "lighter" to LMK scoring
- potentially moves you out of the next kill threshold

In practice:

- Without cleanup: memory stays high → LMK escalates → your app becomes a kill candidate
- With cleanup: your app drops memory early → system can reclaim pressure elsewhere → LMK delay or avoidance

Important nuance: You are not "stopping LMK" — you're making yourself less likely to be selected.

4. Because GC is not designed for resource management under system pressure.
 1. GC is not immediate or deterministic. You cannot control when GC runs `System.gc()` is only a hint (and often ignored or delayed)
 2. GC only frees unreferenced objects. GC does NOT help if:
 - caches still hold references
 - singleton maps still contain data
 - bitmaps are still referenced

So memory pressure remains even after GC.

3. GC doesn't handle "intentional retention". Most app memory issues come from:
 - image caches
 - LRU caches
 - static collections
 - pooled objects

These are designed to persist, so GC won't touch them.

4. `onTrimMemory()` is system-guided, not speculative. GC = reactive, blind cleanup
`onTrimMemory()` = proactive, context-aware signal from Android. It tells you:
"I am under pressure right now — reduce footprint"
That's far more actionable than hoping GC guesses correctly.

Summary

- Add cache clearing at different trim levels (light → aggressive)
- It helps reduce memory pressure, indirectly delaying LMK decisions
- It is preferred over GC because:
 - GC is non-deterministic
 - GC cannot clear retained caches
 - `onTrimMemory()` is a system-level signal, not guesswork

Exercise 10:

Case 1:

1. Answer:

a) Static reference to an Activity prevents Activity from being garbage collected

A static variable:

- belongs to the class, not the instance
- lives for the entire lifetime of the app process

What happens internally : When the activity is created:

LeakyActivity instance → assigned to static variable → stays in memory

Even after `finish()` OR rotation OR back navigation: the Activity cannot be destroyed in memory, because the static field is still referencing it.

Why this is bad: An Activity holds references to: Views, Window, Theme, Resources, Entire UI hierarchy...

So leaking it means: You are leaking a huge object graph, not just the Activity.

b) GC cannot remove objects that are still strongly referenced i.e. GC only removes objects that are not reachable from GC roots. The reference chain is:

Static field → Activity → everything inside it

But static fields are GC roots.

So:

- GC sees the Activity is still reachable
- Therefore it is NOT eligible for garbage collection

Important concept: GC does NOT detect “unused objects”, it only detects “objects that are unreachable”

2. Application Context lives as long as the app, it does NOT hold UI references, safe for static storage.

BUT IMPORTANT: Use application context only when you don’t need UI and you don’t need theming or window access

Case 2:

1. LeakyActivity

a. Why this code causes a memory leak

Root cause: Implicit strong reference from inner class

This part:

```
new Runnable() {
    @Override
```

```

        public void run() { ... }
    }

```

is an anonymous inner class.

What Java actually generates:

It is roughly equivalent to:

```

class AnonymousRunnable implements Runnable {
    final LeakyActivity outer;

    AnonymousRunnable(LeakyActivity activity) {
        this.outer = activity;
    }
}

```

So it holds:

LeakyActivity.this

Which is a Strong reference to the Activity.

The Full reference chain (this is the key) When startTask() runs:

GC Root (Thread) → Thread object → Runnable (anonymous inner class)
 → LeakyActivity (implicit reference) → textView → Entire View hierarchy

This is a reachable object graph.

Why this becomes a leak?

Scenario:

1. User opens Activity
2. Thread starts (5 minutes)
3. User: presses back OR rotates device → Activity should be destroyed
 BUT: Thread is still running and Runnable still holds reference to Activity
 Therefore: Activity is STILL reachable
 → GC cannot collect it

b. Why Garbage Collector cannot fix it

GC rule: GC only collects objects that are NOT reachable from GC Roots

Is Activity reachable?

Yes:

Thread (GC Root)

→ Runnable

→ Activity

So Activity is strongly reachable, Therefore: GC does NOT consider it garbage

Even if Activity is destroyed (lifecycle) and UI is gone

Memory is retained unnecessarily

In summary:

a. Why memory leak occurs:

- Anonymous inner class holds implicit strong reference to Activity
- Thread lives longer than Activity lifecycle
- Activity remains reachable via thread

- b. Why GC cannot fix it:
 - Activity is still reachable from a GC Root (Thread)
 - GC only collects unreachable objects
 - Therefore, Activity is not eligible for collection

2. SafeActivity

Why no memory leak:

- Static inner class removes implicit reference to Activity
- WeakReference does not prevent garbage collection
- Activity becomes unreachable after destruction

Why GC now works:

- No strong references from GC roots
- Only weak reference exists
- GC clears Activity → get() returns null

Safety mechanism: Null checks prevent accessing destroyed Activity

Exercise 11:

1. Memory usage info.
2. RSS: total, PSS: shared divided, USS: private. (See 4)
3. PSS is most accurate. (See 4)
4. Shared memory is divided.

Step 1: Break down memory

Private memory

- A = 30 MB
- B = 30 MB
- C = 30 MB

Shared memory

LibrarySize	Shared by
X 90 MB	A, B, C (3 processes)
Y 60 MB	A, B (2 processes)

Step 2: Compute each metric

USS (Unique Set Size)

Only private memory counts:

- A = 30 MB
- B = 30 MB
- C = 30 MB

Process	USS
A	30 MB
B	30 MB
C	30 MB

Total 90 MB

Shared memory is ignored completely

RSS (Resident Set Size)

Counts full shared memory for every process:

A:

$30 + 90 + 60 = 180 \text{ MB}$

B:

$30 + 90 + 60 = 180 \text{ MB}$

C:

$30 + 90 = 120 \text{ MB}$

Process	RSS
A	180 MB
B	180 MB
C	120 MB

Total 480 MB

Severe overcounting due to duplication of shared memory

PSS (Proportional Set Size)

Shared memory is divided fairly.

Library X ($90 \text{ MB} / 3 = 30 \text{ MB}$ each)

- A: 30
- B: 30
- C: 30

Library Y ($60 \text{ MB} / 2 = 30 \text{ MB}$ each)

- A: 30
- B: 30
- C: 0

Final PSS per process

Proc.	Private	X share	Y share	PSS
A	30	30	30	90 MB
B	30	30	30	90 MB
C	30	30	0	60 MB

Proc.	PSS
A	90 MB
B	90 MB
C	60 MB

Total 240 MB

What this example shows clearly

1. RSS becomes wildly inflated

- True system memory $\approx 150 \text{ MB}$ (private + shared once)
- RSS reports 480 MB (massive overcount)

2. USS ignores too much

- Only shows private memory = 90 MB total
- Misses shared system cost entirely

3. PSS matches reality best

- Total = 240 MB
- Correctly counts:
 - o all private memory
 - o shared memory exactly once (distributed fairly)